Symulator Tomografu Komputerowego (CT)

Skład grupy: **Jan Chojnacki, Albert Łapa**

Contents

1 Cel i zakres projektu **2**

2 Narzędzia i stack technologiczny **2**

3 Metody i algorytmy **3**
3.1 Transformata Radona i algorytm Bresenhama 3
3.2 Filtrowanie (Filtered Backprojection - FBP) 4
3.3 Ustalanie jasności pikseli obrazu wynikowego i normalizacja 4
3.4 Odczyt i zapis DICOM 5

4 Ocena jakości rekonstrukcji **6**
4.1 Przykład działania programu dla dwóch obrazów wejściowych 6
4.2 Metodyka eksperymentów 7
4.3 Wyniki eksperymentów dla poszczególnych parametrów 8
4.4 Wpływ filtrowania (FBP) 10

5 Wnioski **10**

1 Cel i zakres projektu

Głównym celem projektu było zaprojektowanie oraz implementacja interaktywnego symulatora tomografii komputerowej (CT). Aplikacja ma za zadanie odzwierciedlać rzeczywiste procesy zachodzące podczas badania, od skanowania, przez przetwarzanie sygnału, aż po rekonstrukcję obrazu i jego zapis w plikach DICOM. W ramach projektu zaimplementowano dwa modele geometrii wiązki: stożkową oraz równoległą.

2 Narzędzia i stack technologiczny

Projekt zrealizowano w języku **Python**, wykorzystując następujące biblioteki:

- **Streamlit:** Do stworzenia interaktywnego GUI.
- **Numba:** Do kompilacji JIT krytycznych sekcji kodu w celu przyspieszenia zagnieżdżonych pętli w algorytmach transformaty Radona
- **NumPy:** Do operacji na macierzach i obliczeń numerycznych
- **pydicom:** Do generowania, formatowania i zapisu plików DICOM
- **scikit-image oraz Matplotlib:** Do wczytywania i normalizacji obrazów oraz rysowania wykresów

3 Metody i algorytmy

3.1 Transformata Radona i algorytm Bresenhama

Proces powstawania sinogramu opiera się na symulacji pochłaniania promieniowania X. Zastosowano zoptymalizowany algorytm Bresenhama do dyskretnego wyznaczania pikseli leżących na odcinku między emiterem a detektorem. W aktualnej wersji użyto wariantu supercover oraz modelu szerokości detektora (średnia z trzech równoległych linii liczonych Bresenhamem), co ogranicza artefakty takie jak "krzyżyki" na sinogramie. Projekcja pozostaje addytywna: do sinogramu trafia suma osłabień na promieniu.

Zoptymalizowana transformata Radona (fragment)

```python
@njit(cache=True)
def radon_transform(image, xe_idx, ye_idx, xd_idx, yd_idx):
    steps = xe_idx.shape[0]
    n = xe_idx.shape[1]
    sinogram = np.zeros((steps, n), dtype=np.float32)

    for s in range(steps):
        for i in range(n):
            x0 = xe_idx[s, i]
            y0 = ye_idx[s, i]
            x1 = xd_idx[s, i]
            y1 = yd_idx[s, i]
            line_sum, valid_samples = line_integral(image, x0, y0, x1, y1)
            if valid_samples > 0:
                sinogram[s, i] = line_sum
    return sinogram
```

3.2 Filtrowanie (Filtered Backprojection - FBP)

Zwykła odwrotna transformata Radona prowadzi do znacznego rozmycia zrekonstruowanego obrazu. Aby temu zapobiec, zaimplementowano filtrowanie (splot) poszczególnych wierszy sinogramu z maską górnoprzepustową typu Ramp. W aktualnej implementacji używany jest stały rozmiar jądra (21 próbek), zgodny z funkcją filtrującą.

Tworzenie maski filtrującej

```
def create_filter(size):
    kernel = np.zeros(size)
    center = size // 2
    for i in range(size):
        k = i - center
        if k == 0:
            kernel[i] = 1.0
        elif k % 2 != 0:
            kernel[i] = -4.0 / (np.pi**2 * k**2)
    return kernel
```

3.3 Ustalanie jasności pikseli obrazu wynikowego i normalizacja

W trakcie odwrotnej projekcji każdy promień zwiększa jasność pikseli, przez które przechodzi. Następnie wykonywane jest uśrednienie przez liczbę trafień (mapę pokrycia), co stabilizuje obraz końcowy.

Uśrednianie rekonstrukcji i normalizacja do podglądu

```
for y in range(h):
    for x in range(w):
        if hit_count[y, x] > 0.0:
            reconstruction[y, x] /= hit_count[y, x]

def _normalize_for_display(image):
    max_val = float(np.max(image))
    if max_val > 0.0:
        return image / max_val
    return image
```

3.4 Odczyt i zapis DICOM

Odczyt i zapis standardu DICOM realizowany jest biblioteką `pydicom`. Przy odczycie wykorzystywane są dane pikselowe i metadane, a przy zapisie ustawiane są podstawowe tagi pacjenta i parametry obrazu 16-bit.

Odczyt i zapis DICOM (fragmenty)

```python
# Odczyt DICOM
dataset = pydicom.dcmread(uploaded_file)
pixel_data = dataset.pixel_array.astype(np.float32)
input_image = preprocess_image(pixel_data)

# Zapis DICOM
ds = FileDataset(None, {}, file_meta=file_meta, preamble=b"\0" * 128)
ds.PatientName = patient_name
ds.PatientID = patient_id
ds.StudyDate = _normalize_study_date(study_date)
ds.ImageComments = comments
ds.BitsStored = 16
ds.BitsAllocated = 16
ds.PixelData = pixel_data.tobytes()
```

4 Ocena jakości rekonstrukcji

4.1 Przykład działania programu dla dwóch obrazów wejściowych

Poniżej przedstawiono dwa obrazy wejściowe (`CT_ScoutView.jpg` oraz `Shepp_logan.jpg`) wraz z rekonstrukcją (zarówno bez, jak i z filtrowaniem).

Figure 1: Przykład działania dla `CT_ScoutView.jpg`: obraz oryginalny oraz rekonstrukcje bez filtra i z filtrem.

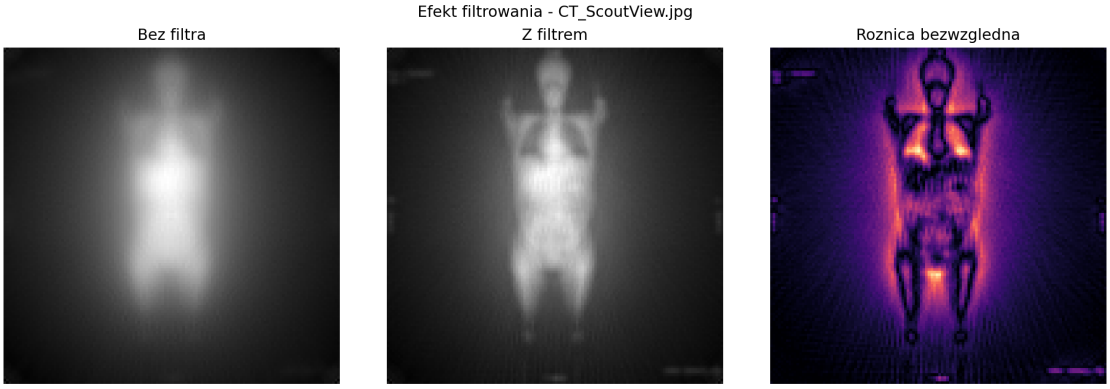

Figure 2: Efekt filtrowania dla `CT_ScoutView.jpg`.

Figure 3: Przykład działania dla `Shepp_logan.jpg`: obraz oryginalny oraz rekonstrukcje bez filtra i z filtrem.

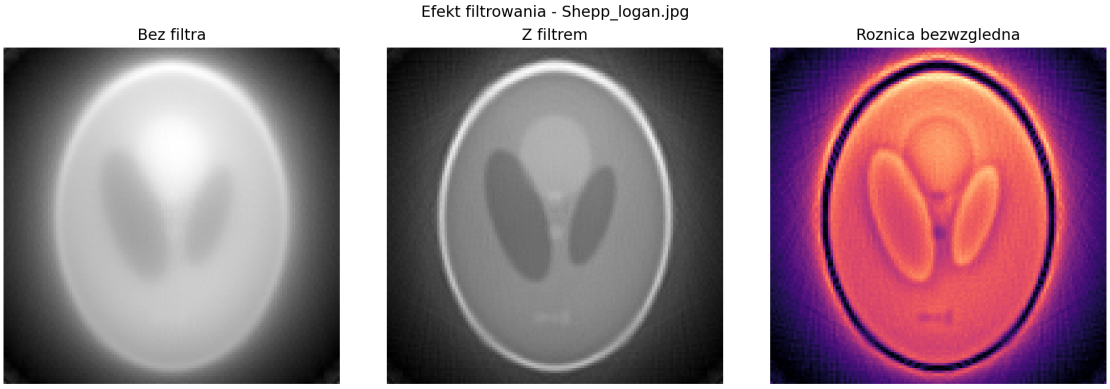

Figure 4: Efekt filtrowania dla `Shepp_logan.jpg`.

4.2 Metodyka eksperymentów

Jako miarę jakości przyjęto błąd średniokwadratowy (RMSE). W implementacji `calculate_rmse()` oba obrazy są najpierw niezależnie normalizowane do zakresu $[0, 1]$, a następnie liczona jest wartość RMSE. Dzięki temu porównanie jest odporne na różnice skali intensywności pomiędzy obrazem wejściowym i rekonstrukcją.

Obliczanie RMSE w projekcie

```python
def calculate_rmse(img1, img2):
    i1 = (img1 - np.min(img1)) / (np.max(img1) - np.min(img1) + 1e-8)
    i2 = (img2 - np.min(img2)) / (np.max(img2) - np.min(img2) + 1e-8)
    return np.sqrt(np.mean((i1 - i2) ** 2))
```

4.3 Wyniki eksperymentów dla poszczególnych parametrów

Eksperyment wykonano na jednym obrazie: `CT_ScoutView.jpg`. Dla wariantów, w których dany parametr nie był zmieniany, przyjęto wartości domyślne: 180 detektorów, 180 skanów, rozpiętość wachlarza 180 stopni.

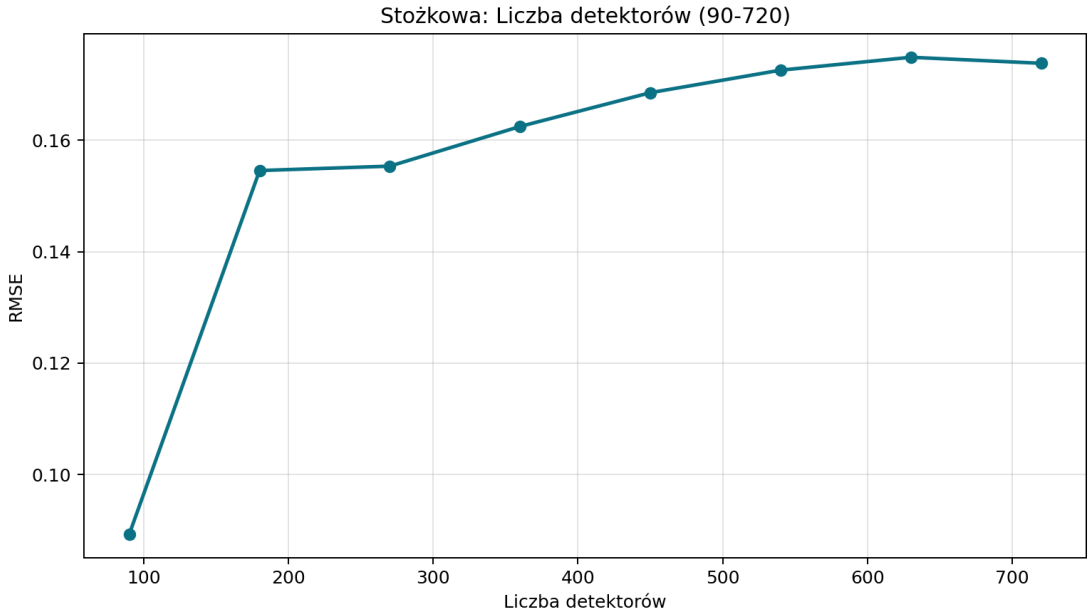

Figure 5: Zależność RMSE od liczby detektorów (90..720, krok 90).

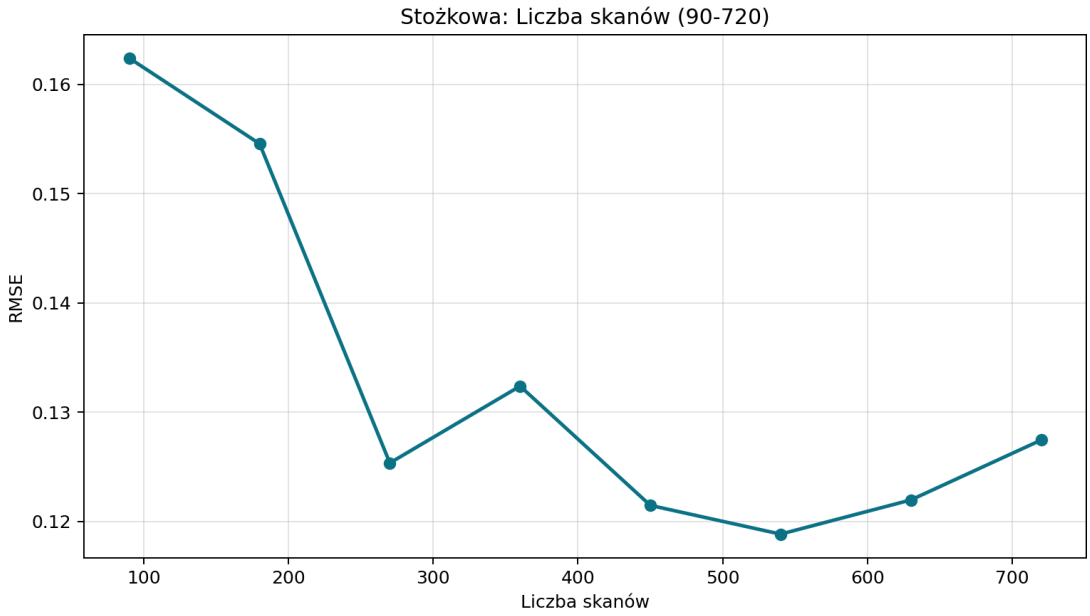

Figure 6: Zależność RMSE od liczby skanów (90..720, krok 90).

Figure 7: Zależność RMSE od rozpiętości wachlarza (45..270, krok 45).

Komentarz do przebiegów RMSE Najniższy błąd dla zmian liczby detektorów uzyskano przy najniższej wartości 90 (RMSE = 0.0893). Dla liczby skanów minimum pojawiło się przy 540 (RMSE = 0.1188), natomiast dla rozpiętości wachlarza przy jej maksymalnej wartości (270 stopni - RMSE = 0.1283). We wszystkich trzech przypadkach przebiegi nie są ściśle monotoniczne.

Table 1: Najlepsze wyniki RMSE dla poszczególnych eksperymentów.

Geometria	**Eksperyment**	**Najlepszy parametr**	**Najlepsze RMSE**
Stożkowa	Liczba detektorów (90-720)	90	0.0893
Stożkowa	Liczba skanów (90-720)	540	0.1188
Stożkowa	Rozpiętość wachlarza (45-270)	270	0.1283
Równoległa	Liczba detektorów (90-720)	180	0.1283
Równoległa	Liczba skanów (90-720)	720	0.1207
Równoległa	Rozpiętość równoległa (% przekątnej: 50-200)	150	0.1085

4.4 Wpływ filtrowania (FBP)

Testy przeprowadzono dla parametrów: 360 detektorów, 360 skanów, rozpiętość 270 stopni. Zastosowanie filtra splotowego znacząco poprawia percepcję krawędzi i obniża RMSE dla obu analizowanych obrazów.

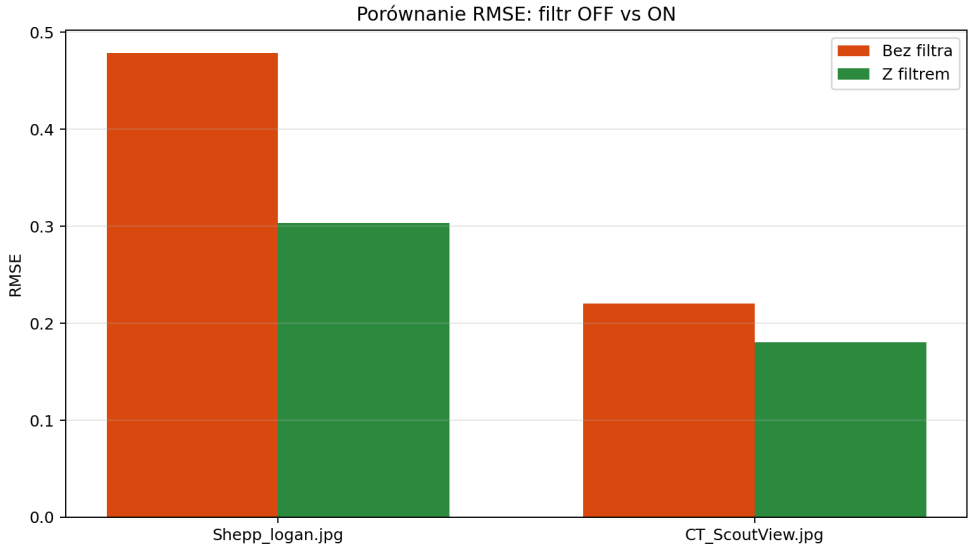

Figure 8: Porównanie RMSE dla rekonstrukcji bez filtra i z filtrowaniem.

Table 2: Porównanie błędu RMSE z filtrowaniem i bez filtrowania.

Testowany obraz	RMSE bez filtra	RMSE z filtrem
`CT_ScoutView.jpg`	0.2199	0.1795
`Shepp_logan.jpg`	0.4784	0.2887

5 Wnioski

1. **Znaczenie rozdzielczości:** Zmiana liczby detektorów oraz liczby skanów istotnie wpływa na RMSE, ale zależność nie jest monotoniczna. W badanym zbiorze najlepsze wyniki pojawiają się dla określonych punktów pracy (np. 90 lub 180), a niekoniecznie dla wartości maksymalnych.

2. **Konieczność filtrowania:** Analiza wizualna oraz wartości RMSE potwierdzają bezwzględną konieczność stosowania filtrowania (FBP). Naiwna wsteczna projekcja nie nadaje się do diagnostyki ze względu na dominujące niskie częstotliwości rozmywające detale.

3. **Optymalizacja obliczeń:** Wykorzystanie biblioteki Numba okazało się kluczowe dla zachowania interaktywności symulatora.